ABSTRACT

Parkinson's disease (PD) is a neurodegenerative disorder that affects nervous system and the root cause of it is falling rates of dopamine levels in the forebrain. At present there is no conclusive result for this disease. Early diagnosis helps in minimizing the risk of Parkinson's disease. The symptoms of early stage include monotony and distortion of speech. Machine learning based approaches helps to automatically detect the disorder and analyze the performance of different combinations by using speech dataset taken from the UCI (University of California, Irvine) machine learning repository. It is having sequence mechanism of data collection, feature selection, train a model, and model prediction the proposed approach is evaluated using XGBOOST technique (Supervised learning algorithm) on the speech dataset which achieves maximum accuracy and low error rate when compared to Random Forest, Naive Bayes, decision tree, KNN classifiers for prediction of Parkinson's disease. Our project contributes for the development of medical technologies this venture might also assist the patient as well as the scientific assistance to come upon parkinson's in advance with early treatment. So, patients can begin with modern-day-remedies earlier than the damaging progression of disorder.

Contents

CHAPTERS	PAGE NO
LIST OF FIGURES	**i**
LISTOF ABBREVIATIONS	**ii**
1. INTRODUCTION	**1**
1.1 Purpose And Objective	2
1.2 Existing And Proposed System	5
1.3 Scope Of Project	6
2. REQUIREMENTS	
2.Requirments Specifications	7
2.1Functional Requirements	7
2.2Non Functional Requirements	8
2.3 Constraints And Prerequistite	9
3. ANALYSIS	
3.1Use Case	10
3.1.1 Use Case Of Diagram	11
3.1.1.1 Flow Of Events	12
4. DESIGN	
4.1 Description	13
4.2 Architecture	16
4.3 Class Diagrams	17
4.4 Sequence Diagram	18
5. METHODOlOGY	
5.1 Technologies Used	23
5.2 Modules Description	25
5.3 Algorithm	31
6. IMPLEMENTATION	
6.1 Sample Code	37
6.2 Userscreens	39

7. TEST METHODOLOGY 43
8. CONCLUSION 44
9. .REFERNCES 45

List Of Figures

FIGURE NO	NAME OF THE FIGURE	PAGE NO
Fig 4.2.1	System Architecture	15
Fig 4.3.1	Data Flow Diagram	19
Fig 4.3.2	Use Case Diagram	21
Fig 4.3.3	Class Diagram	22
Fig 4.3.4	Squence Diagram	23
Fig 4.3.5	Activity Diagram	25
Fig 5.1	Importing Libraries And Loading Data	29
Fig 5.2	Visualization Of Record	31
Fig 5.5	Accuracy,Precision,F1 Score	33
FIg 5.6	Confusion Matrix	33
Fig 5.11	Index.html	36
Fig 5.13	Results.html	37
Fig 5.14	App.py	38
Fig 5.15	App.py(Continued)	38
Fig 6.1	User Input Values	47
Fig 6.2	Predicted Output Of A Healthy Person	48
Fig 6.3	Predicted Output Of A Healthy Person	49

List Of Abbrevations

TERMS	DESCRIPTION
Xg Boost	Extreme Gradient Boosting
ANN	Artficial Neural Network
CNN	Convoltional Neural Network
DFA	Getrended Fluctuation Analysis
DDA	Differential Data Analysis
HNR	Harmonics To-Noice Ratio
KNN	K-Nearest Neighbour
L1	Lasso Regulization
ML	Machine Learning
SVM	Support Vector Machone
UCI	University Of California,Irvine
UML	Unified Modelling Language

1.INTRODUCTION

Parkinson's disease (PD) is a progressive neurodegenerative disorder that primarily affects movement due to the gradual loss of dopamine-producing neurons in the brain, particularly in the substantia nigra. It is estimated that around 10 million people worldwide suffer from Parkinson's, with most cases occurring in individuals over 60. However, about 10% of cases develop before the age of 50, a condition known as Young-Onset Parkinson's Disease. Studies have shown that men are 1.5 times more likely to develop the disease than women. The exact cause of PD remains unknown, but genetic mutations, environmental factors, head trauma, and lifestyle choices may contribute to its onset.

The primary symptoms of Parkinson's are classified into motor and non-motor categories. Motor symptoms include tremors, bradykinesia (slowness of movement), muscle rigidity, and postural instability, while non-motor symptoms include loss of sense of smell, sleep disturbances, depression, cognitive impairment, and speech difficulties. Parkinson's is difficult to diagnose, as there is no definitive laboratory test, and its early symptoms often resemble those of other neurological disorders. By the time symptoms appear, a significant percentage of dopamine-producing neurons may already be lost. Current diagnostic methods rely on clinical observations, making early detection challenging.

With advancements in technology, machine learning (ML) is being explored as a tool to detect Parkinson's in its early stages. ML models utilize clinical data, speech recordings, movement tracking, and imaging techniques to identify patterns associated with the disease. Since voice changes occur in nearly 80-90% of PD patients, speech analysis has become a promising approach for automated detection. Preprocessing techniques help clean the data, followed by feature extraction methods that analyze key indicators such as vocal tremor, pitch variation, and jitter. Various classification algorithms, including XGBoost, Support Vector Machines (SVM), and deep learning models, are then applied to differentiate PD patients from healthy individuals. The effectiveness of these models is evaluated using accuracy, precision, recall, and ROC-AUC scores.

1.1 PURPOSE AND OBJECTIVES

The main purpose of this project is identification of Parkinson's disease at early stages using Machine learning model

This project can be implemented in the following steps:

- Gathering the data
- Data preparation
- Data wrangling
- Data analysis
- Training the model
- Testing the model
- Deployment of model

XGBoost (Extreme Gradient Boosting) is a powerful and efficient machine learning algorithm that builds upon the principles of gradient boosting to enhance model performance. It is widely used for classification and regression tasks due to its speed, scalability, and ability to handle missing values and complex datasets. In this project, XGBoost is chosen for its ability to optimize prediction accuracy while maintaining computational efficiency. The algorithm works by creating an ensemble of decision trees, where each tree corrects the errors of its predecessor, ultimately improving the model's precision in detecting Parkinson's disease.

The primary advantage of using XGBoost in Parkinson's detection is its capability to handle structured medical data, including speech features, movement patterns, and patient history. The model processes patient input data, compares it with previously collected cases, and determines the likelihood of Parkinson's disease. The algorithm's feature selection mechanism ensures that only the most relevant attributes—such as voice tremors, pitch variability, and movement irregularities—are used for classification, reducing noise and enhancing reliability.

This project plays a crucial role in the advancement of medical technologies, particularly in the field of early disease detection. Parkinson's is often diagnosed at a late stage due to the subtlety of its initial symptoms, which can lead to delayed treatment and worsening of the condition. By integrating machine learning techniques, this system aims to provide a proactive solution, allowing healthcare professionals to detect Parkinson's in its early stages. Early diagnosis enables patients to begin treatment sooner, which may include medication, physical therapy, and lifestyle modifications to slow disease progression.

1.2 EXISTING AND PROPOSED SYSTEM

Existing System

- In the classification techniques , popular classifiers as KNN ,decision tree ,Random Forest doesn't give maximum accuracy as compared to the XGBOOST
- From existing survey the efficiency of various classifiers are as follows:
- KNN:82.05%
- DecisionTree:89.74%
- Random Forest:92.30%

Disadvantages

- Relatively Less Speed
- relatively much less accuracy

Proposed System

- The proposed approach uses XG Boost algorithm.
- XG Boost is a Boosting technique in Ensemble Learning.
- XG Boost is an enhancement of Gradient Boosting algorithm
- Here, decision trees are generated in sequential order.
- This algorithm is highly efficient in training the model and predicts the result in such a way that it maintains the precision and accuracy of the system

Advantages:

a) Execution rate

b) Overall efficiency

XGBOOST ALGORITHM

- Algorithm we used right here is XG Boost algorithm
- XG Boost is a Boosting technique in Ensemble Learning. Ensemble learning includes a set of predictors which use multiple models for providing better prediction accuracy
- "XG Boost" is an enhancement of "Gradient Boosting algorithm"
- Here, decision trees are created in "sequential order".
- "Weights" plays crucial role in "XG Boost". Weights are allocated to all independent variables which are then fed into decision tree which predicts output

- The weights of variables predicted incorrectly by tree are increased and these variables are then fed into second decision tree. These individual classifiers then group to give a strong, more precise model
- It works on different prediction problems like CART, ranking, user-defined problems

➢ WHY XGBOOST?

MAIN FEAUTURES OF GRADIENT BOOSTING

- Parallel tree building –the sequential tree building done in parallelized implementation by
- interchanging the loops which enumerates the leaf nodes of tree.
- Regularization- XG Boost has the potential to correct complicated models through “L1, L2
- regularization”. Regularization helps by avoiding over fitting
- Tree pruning using depth-first approach XG Boost uses 'max_depth' parameter and starts pruning decision tree backwards.
- This "depth-first" approach significantly increases the computational performance.



- Consideration of cache and out-of-core calculation-this optimizes the memory of the disk

- While making sure there is no continuous access of hardware and better usage of hardware and memory space by storing data in blocks.
- Efficient handling of missing data-actually it is dealing with sparse data while incorporate With sparse wise split algorithm which make sure to handle different sparse data and its patterns
- Built-in cross-validation capability- it has this built in method which at every iteration, the need to explicitly program this search and to mention precise number of boosting iterations needed taking single run

1.3 SCOPE OF THE PROJECT

Parkinson disease is a neurodegenerative disorder that mostly presents in later life with generalized slowing of movements (bradykinesia) and at least one other symptom of resting tremor or rigidity. Other associated features are a loss of smell, sleep dysfunction, mood disorders, excess salivation, constipation, and excessive periodic limb movements in sleep (REM behavior disorder). It is estimated that Parkinson disease affects at least 1% of the population over the age of 60. The disorder is associated with the loss of dopaminergic neurons in the substantia nigra and the presence of Lewy bodies. Most cases are idiopathic. Only about 10% of cases have a genetic cause, and these cases are seen in young people. The disorder has a slow onset but is progressive. Tremor is often the first symptom and later can be associated with bradykinesia and rigidity. Postural instability is usually seen late in the disease and can seriously impact the quality of life. Also of importance is the presence of autonomic symptoms that may precede the motor symptoms in some patients. The diagnosis in most patients is based on history and clinical presentation. SPECT scans can be performed in doubtful cases or to rule out other neurological disorders.

2.REQUIRMENTS

2.1 SOFTWARE REQUIREMENTS SPECIFICATION

2.1.1FUNCTIONAL REQUIREMENTS

- They describe the functionality of system services
- They depend on numbers of users expected to use a system, type of software used and type of system used in system building
- These are simple service statements that define what a system should provide as input and how that system should react to the input and it should behave in different situations
- These can be further divided into 2 types such as

a) **Functional User Requirements:** These are the statements that define what a system should do. Generally, these statements are complex in nature.

b) **Functional System Requirements:** These are also high level statements that describe the services provided by system in detailed manner.

- The nature of the requirements is complete and consistent

2.1.2.NON FUNCTIONAL REQUIREMENTS

- They describe and define the constraints and properties of the system. Some examples of constraints are System representations, Input and Output device capabilities. Some examples of properties are Response time, Reliability, Storage capacity and so on
- They generate new requirements and sometimes restrict old or existing requirements based on prevailing conditions
- A failure in non-functional requirements may affect the entire system architecture
- A single non-functional requirement can generate single or multiple functional requirements
- They also define process requirements such as programming language, case system and development method
- There are 3 types of non-functional requirements namely

a) **External Requirements:** The factors which are necessary for the system that arise as an external need. Examples of external requirements are Ethical requirements, Legislative requirements, Interoperability requirements etc.

b) **Organisational Requirements:** The factors that arise as a consequence of a particular organisation standards, procedures and policies. Development requirements, Environment requirements and so on are examples of organisational requirements.

c) **Product Requirements:** These requirements define how a delivery product should behave in a particular situation. Some of the examples of this requirements are Efficiency Requirements, Security Requirements, Usability Requirements and Dependability Requirement.Describe user- visible aspects of the system that are not directly related to the functional behavior of the system. Non- Functional requirements include quantitative constraints, such as response time or accuracy.

- Accuracy
- Security
- Maintainability
- Reliability
- Scalability
- Performance
- Flexibility

2.3 CONSTRAINTS AND PREREQUISITES

Constraints:

While the system offers efficient plant disease detection, certain limitations exist:

Internet Dependency: Since the system is web-based, it requires an active internet connection to process images, fetch weather data, and access chatbot features.

Image Quality Requirements: The accuracy of disease detection depends on clear, high-resolution images. Blurry or low-light images may reduce model accuracy.

Dataset Limitations: The CNN model is trained on the PlantVillage dataset, which may not cover all plant species and rare diseases. Continuous updates are needed for better accuracy.

Limited Real-Time Analysis: While the system provides instant disease detection, it does not currently offer real-time monitoring using IoT devices. Future enhancements could include IoT integration for live plant health tracking.

Multilingual Support in Chatbot: While the chatbot can process standard queries in English, multi language processing is limited, requiring further NLP model training.

Prerequisites

Before setting up the plant disease detection system, the following prerequisites are required:

1. Technical Knowledge

Basic understanding of Python programming.

Familiarity with Machine Learning concepts, especially Convolutional Neural Networks (CNNs).

Basic knowledge of Flask framework for web development.

Understanding of image processing techniques.

2. Software Requirements

Python 3.x installed on your system.

A code editor such as Visual Studio Code, PyCharm, or Jupyter Notebook.

A web browser (Google Chrome, Mozilla Firefox, etc.) to access the web interface.

Stable internet connection for accessing OpenWeather API.

3. OpenWeather API Key

Sign up at OpenWeather to get a free API key required for fetching real-time weather data.

4. Hardware Requirements

Minimum 8GB RAM (16GB recommended).

Decent CPU processor.8

Installations:

Install Required Python Libraries

Use the following command to install all required libraries:

Fig 2.3.1 Installation of libraries

Flask: For web application development.

PyTorch & TorchVision: For deep learning-based plant disease classification.

Pandas & NumPy: For data handling and numerical computations.

Pillow (PIL): For image processing.

Requests: For connecting to the OpenWeather API.

3.SYSTEM ANALYSIS

3.1 Use Case Model

3.1.1 Use Case Diagram

A use case diagram is a graphical depiction of a user's possible interactions with a system. A use case diagram shows various use cases and different types of users the system has and will often be accompanied by other types of diagrams as well. The use cases are represented by either circles or ellipses. To depict a system's dynamic behavior, use case diagrams are often employed. Using use cases, actors, and their interactions, it captures the functionality of the system. A system or subsystem of an application's necessary duties, services, and operations are modelled. It shows a system's high- level functionality as well as how a user interacts with that system.

In this use case diagram we have user who gives the voice signal as input to system the system analysis the voice and based on trained data it predict the result.

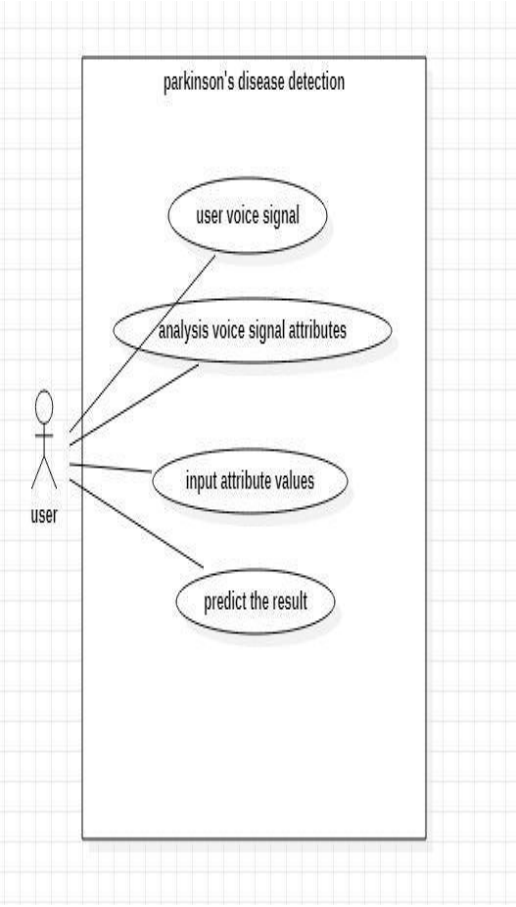


Fig3.1.1 use case diagram

3.1.1.1 FLOW OF EVENTS

A data-flow diagram is a visual representation of how data moves through a system or a process (usually an information system). The data flow diagram also shows the inputs and outputs of each entity as well as the process itself. A data-flow diagram lacks control flow, loops, and decision-making processes. With a flowchart, certain operations based on the data can be depicted

The flowchart can be used to understand how the data flows in this project .input data is given in the form of csv file it preprocess the data then train the data and compare the data based on previous data then the result is declared as parkinson's disease or No parkinson's disease.

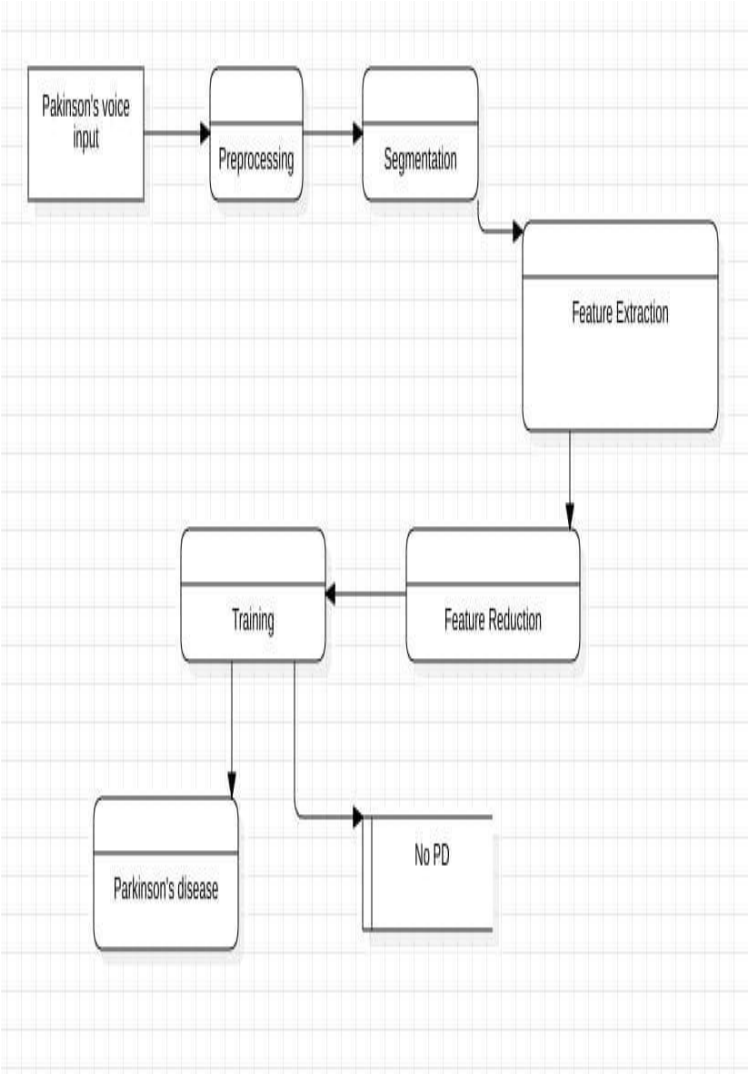


Fig 3.1.1.1 Flow Of Events

4.DESIGN

4.1 DESCRIPTION

System design is the process of creating a system's architecture, parts, and interfaces to ensure that it satisfies the needs of its users.

Thus, in order to examine the design of this project, we first go through the specifics of establishing the concept of news detection through a few fundamental modules that would clearly describe the workings of the system that would come from the development.

MACHINE LEARNING

Machine learning (ML) is an area of artificial intelligence (AI) that enables computers to "learn" for themselves over time from training data and develop without explicit programming. Data patterns can be found by machine learning algorithms, which can then use this information to learn and develop their own predictions. Algorithms and models used for machine learning, in essence, gain experience.

In contrast, machine learning is a process that is automated and gives computers the ability to solve issues with little to no human involvement and make decisions based on prior experiences.

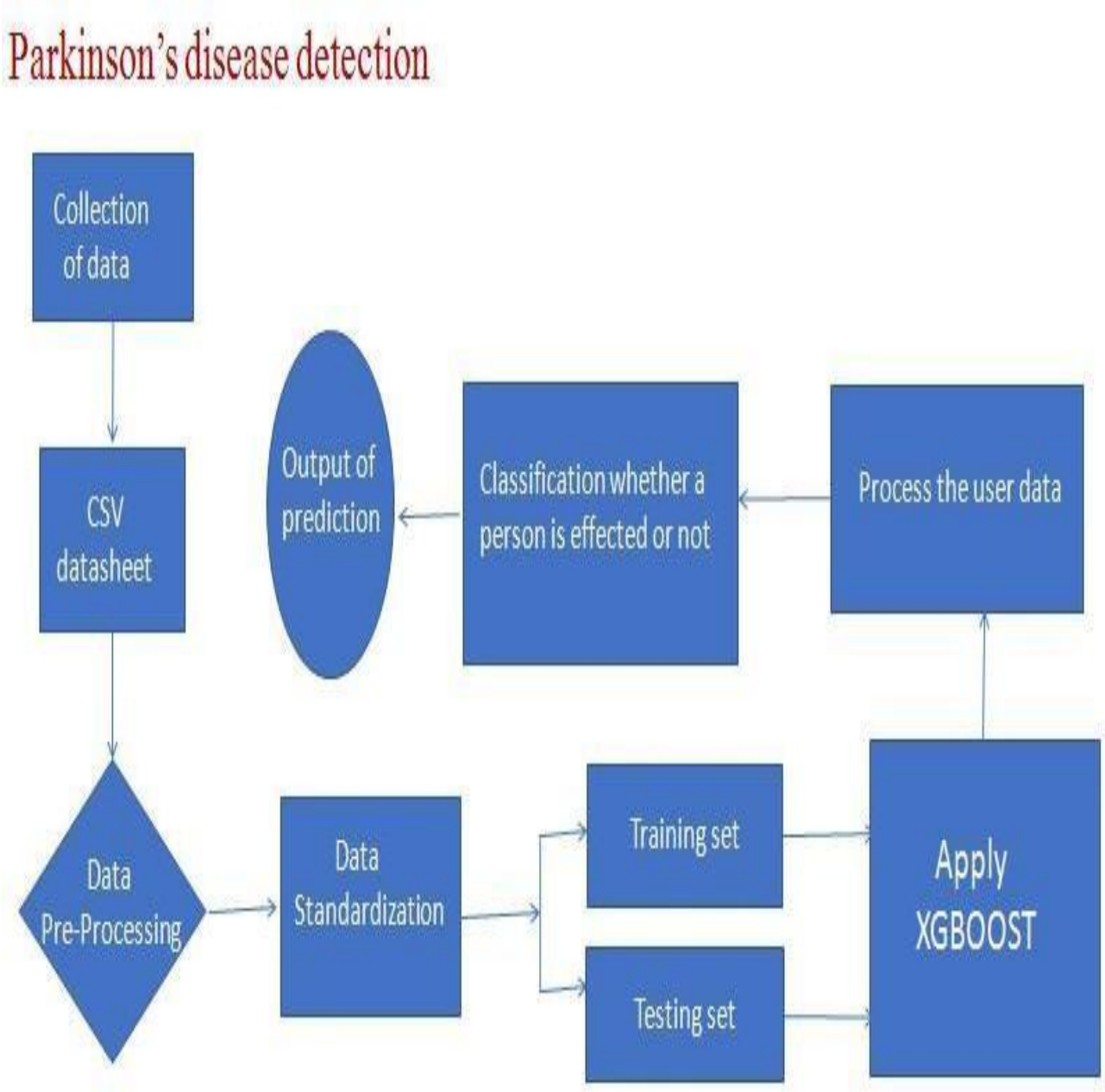
While machine learning and artificial intelligence are frequently used synonymously, they are actually two distinct ideas. Machine learning, a subset of AI that enables intelligent systems to autonomously learn new things from data, is what allows intelligent systems to make decisions, gain new abilities, and solve problems in a way that is similar to people. AI is the more general notion..

Machine learning methods are used in image classification to examine the existence of objects in a information and classify it.

12

4.2.1 ARCHITECTURE

This concept is similar to discussions regarding machine learning and its algorithms. For generalization and to examine its specifics.



4.2.1 : System Architecture

- Step-1: Collecting the attribute values of voice signal using praat software
- Step-2: Load the dataset and check for null values and remove unnecessary data-data preprocessing
- Step-3: Noise free data values are scaled to a particular range for easy processing and analyzing
- Step-4: The data is partitioned into "training set (80% of data)" and "test set (20%)"
- Step-5: The "xgboost algorithm" keeps on training the model based on the input until it produces the required output
- Step-6: The algorithm stops training itself as it produces the desired output followed by testing phase to fix any bugs or errors found during the model building stage
- Step-7: After training the model we give the user input through the html webpage and this trained model helps in predicting the output of given input
- Step-8: The output is displayed in the webpage showing whether the person contains parkinson's disease or not

The architecture of a Parkinson's Disease using XG BOOST typically involves several key components and steps. They are:

DATA COLLECTION:

- The initial phase in the ML process is data gathering. The aim of this phase is to discover and obtain all data-related issues. Here, we need to recognize different data sources, as data can be taken from various resources such as files, database, internet etc. It is a crucial stage in life cycle. The output's performance can be decided by quantity, quality of data collected. The more the data, the greater the accuracy of prediction
- It involves the following tasks
  - finding different data sources
  - gathering data
  - Integration of data acquired from various sources
- We get a coherent set of data by performing the above tasks which is called as a dataset, this dataset is used in next phases

DATA PREPARATION:

- We must ready the data for further processing after it has been gathered. Data preparation entails keeping our data in an appropriate location and making it ready to use in "machine learning training"
- Here, first, we position all data together, and then randomize the ordering of data
- In this phase it involves two processes

a) Data exploration: We need to understand the nature of data that we ought to work with. We must comprehend data's features, format, and quality. An effective outcome is the result of a greater knowledge of data. All correlations, general patterns, and outliers are found here.
b) Data pre-processing: Now the subsequent step is pre-processing of data for its analysis

DATA WRANGLING

- In this process cleaning and extracting of useful information from raw data takes place
- It involves cleaning the data, choosing variable for utilization, changing the data into a suitable layout for analysis is done
- It is an essential step in complete process. In order to overcome the quality issues, data must be cleaned
- It is not essential that data we gathered be of constant use to us, as some might not be useful. Data acquired in real-world applications might have a variety of difficulties, including

a) missing values
b) duplicate data
c) invalid data
d) noise

- We can clean data using different "filtering techniques". The aforesaid concern should be recognized and resolved as they can have a damaging impact on quality of the final product

DATA ANALYSIS

- The data is cleaned and is prepared to be analysed. This stage involves

a) Selection of analytical techniques
b) Building models
c) Evaluation of end result

- The purpose of this is to produce a ML model that analyses data using various analytical techniques and evaluates result
- It starts with identification of problem type, followed by selection of a ML approach like classification, regression, clustering etc. followed by construction of model using prepared data, and finally evaluation of model
- We take data and use ML algorithms to build model

TRAIN MODEL

- Next step is training the model, in this step we train our model to enhance its overall performance and obtain better results to the problem

- Datasets are used for training the model by using various machine learning approaches
- Training a model is important so that it can analyze various patterns, rules, and features

TEST MODEL

- We check our model once it is skilled on a specific dataset. We also test for accuracy of our model in this step by feeding it a test dataset
- The % accuracy of model is decided by testing it against the project or problem's requirements or conditions

DEPLOYMENT

- The last phase in “machine learning life cycle” is deployment, which incorporates setting model into action in a real-world system
- We set-up the model in real system if the set model produces a correct output which meets our conditions at a reasonable pace. Still, before launching the design, we are going to see if it improves standard performance by using the accessible data
- The deployment step is just like finishing a design’s final document

4.3.3 Class Diagram

Class diagrams are the main building block of any object-oriented solution. It displays a system's classes, along with each class's properties, operations, and relationships to other classes. Most modelling tools include three elements to a class. Name is at the top, followed by attributes, then operations or methods, and finally, methods. Classes are linked together to generate class diagrams in a complex system with numerous related classes. Various sorts of arrows represent different relationships between classes.

The class diagram has four main classes physician ,patients ,database ,form.the patients fill the form the management stores the data of the patient then the patient meet the physician he will prescribe the instructions based on patient data.

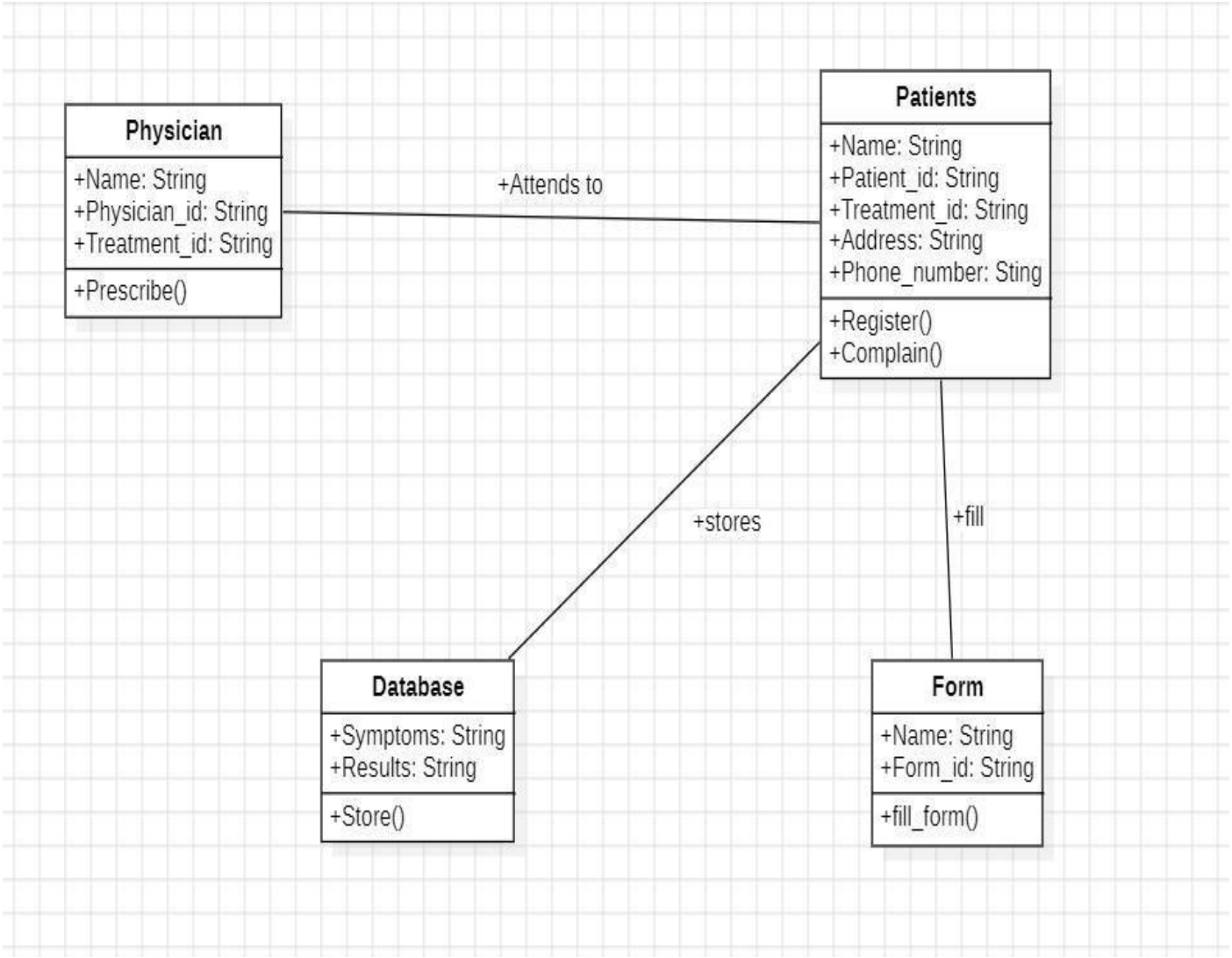


Fig. 4.3.3 Class Diagram

4.3.4 Sequence Diagram

In UML, sequence diagrams display how and in what order certain items interact with one another. It's crucial to remember that they depict the interactions for a certain circumstance. The interactions are depicted as arrows, while the processes are portrayed vertically. The objective of sequence diagrams and their fundamentals are explained in this article. To understand more about sequence diagrams, you may also look at this comprehensive tutorial.

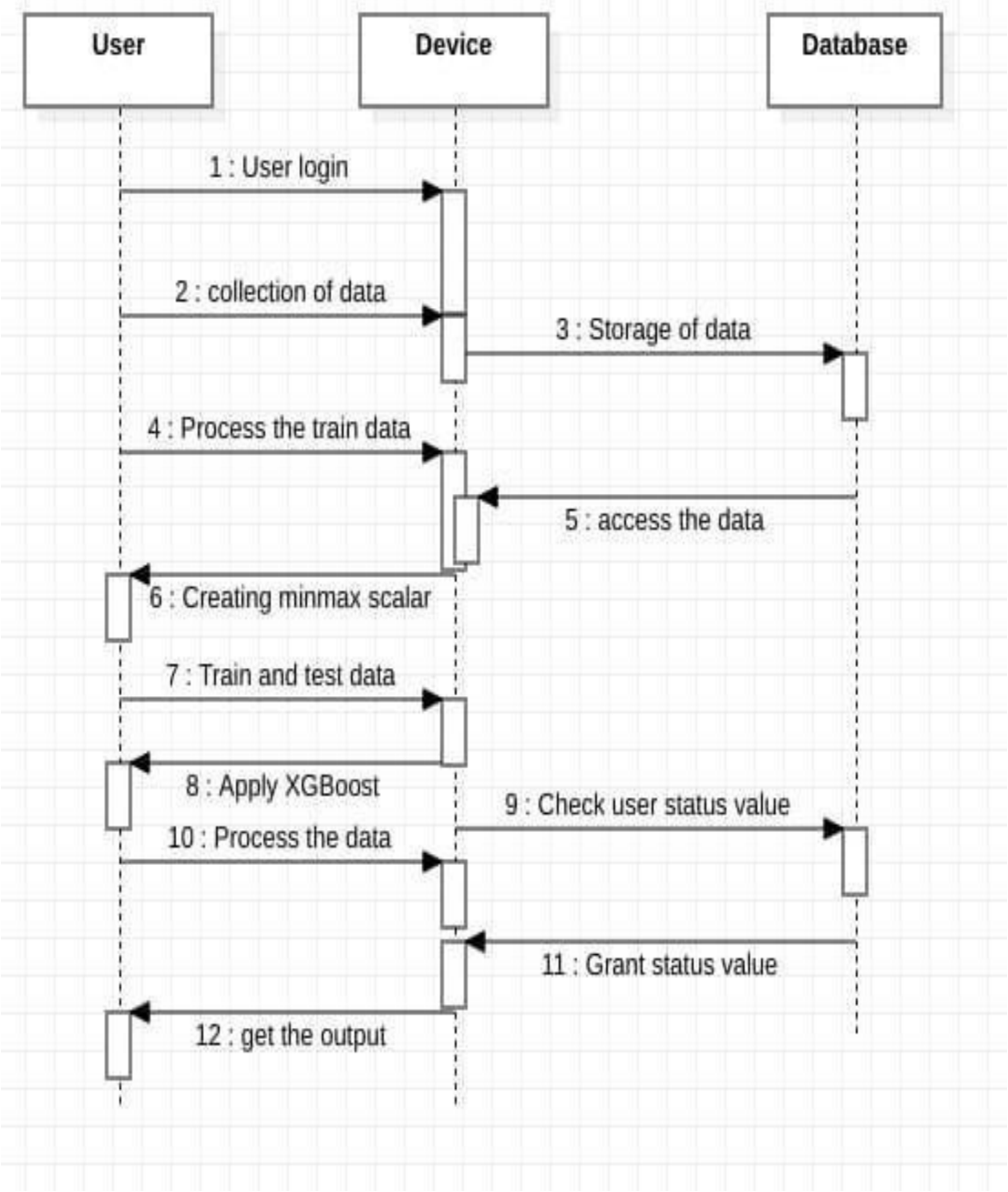


Fig. 4.3.4 sequence Diagram

5.METHODOLOGY

5.1 TECHNOLOGIES USED

PYTHON

Python is a programming language, Guido van Rossum, a Dutch software programmer, devised Python to overcome some of the difficulties he identified in existing computer languages at the time. Python is interpreted, high-level general-purpose programming language. Guido van Rossum developed Python, which was first released in 1991. It was created with a design philosophy that promotes readability of code and a syntax that allows programmers to express concepts with fewer lines of code and more whitespace. It has structures that allow for clear programming at both local and large sizes. Python features a dynamic type of system and automatic memory management. It features a big and extensive standard library and supports several programming paradigms, including object oriented, imperative, functional, and procedural. Numerous operating systems have Python interpreters. C Python, like nearly all of its alternative implementations, is open-source software with a community-based development strategy. The Python Software Foundation, a non-profit organization, oversees C Python.

Python is a high level interpreted ,general purpose programming language . Its design philosophy emphasizes code readability with the use of significant indentation. Guido van Rossum began working on python in the late 1980s as a successor of the ABC programming language and first released it in 1991 as python 0.9.0 python 2.0 was released in 2000 and introduced new features such as list comprehensions, cycle-detecting garbage collection, reference counting, and Unicode support. Python 3.0 released in 2008,was a major revision that is not completely backward compatible with earlier versions.

Python 2 was discovered with version 2.7.18 in 2020 .Python consistently ranks as one of the most popular programming languages.Python programming language is a high level language that can be characterized by all of the following buzzwords:

Simple

Scripting

Object oriented

Portable

Distributed

High performance

Interpreted

Rapid prototyping

Steering

Rapid Application Development

YOU CAN USE PYTHON FOR ANYTHING

One significant advantage of learning Python is that it is a general-purpose language that can be applied in a large variety of projects. Below are just some of the most common fields where Python has found its use:

- Data science
- Scientific and mathematical computing
- Web development
- Computer graphics
- Basic game development
- Mapping and geography (GIS software)

PYTHON IS WIDELY USED IN DATA SCIENCE

Python's ecosystem is growing over the years and it's more and more capable of the statistical analysis. It is the best compromise between scale and sophistication. Python places a premium on productivity and readability. Python is used by programmers that want to delve into data analysis or apply statistical techniques. For data visualization, machine learning, natural language processing, complicated data analysis, and more, Python scientific packages abound. Python is a fantastic tool for scientific computing because of all of these features, and it's a good alternative to commercial software like MATLAB. The most popular libraries and tools for data science are:

ESSENTIAL PYTHON LIBRARIES UESED IN THIS PROJECT

- **pandas**: pandas-"panel data," which refers to multidimensional structured data sets. It is a data- handling and-analysis library. Data structures and procedures for manipulating numerical tables and time series are included in this collection, pandas library is also known as "Python Data Analysis Library".

- **numpy** : numpy is a python package for array processing. It is used for working with multidimensional array object as well as utilities for manipulating them. This is a foundational python package that adds support for big, multi-dimensional arrays and matrices, as well as a vast library of high-level mathematical methods to operate on these arrays.

- **matplotlib:** matplotlib is a Python 2D plotting package that generates high-quality figures in a range of hardcopy and interactive formats across platforms. Plots, histograms, power spectra, bar charts, error charts, scatter plots, and more are all possible with matplotlib.
- **scikit-learn**: scikit-learn are a machine learning library. It includes svm, random forests, gradient boosting, k-means, and DBSCAN and also different classification, regression, and clustering techniques, and it is designed to work with python numerical and scientific libraries like numpy and scipy.
- **csv:** csv module comes with python software. CSV stands for comma separated values. The csv format is the most common import and export format for spreadsheets and databases. The csv module implements classes to read and write tabular data in csv format.
- **flask**: Flask is a micro web framework written in Python. It is referred to as a micro framework because it does not necessitate using any specific tools or libraries. For example, it doesn't have a database abstraction layer, form validation, or other components that rely on third-party libraries to do typical tasks. On the other hand, extensions can be used to add application functionalities as if they were built into Flask itself. For example, object-relational mappers, form validation, upload handling, different open authentication protocols, and other framework-related tools all have extensions. Pinterest, LinkedIn applications are examples that employ the Flask framework
- **pickle**: pickle is a python module which is used for serializing and de-serializing the python objects. For loading the saved model, we need to pass the pickled model into the pickle load() function and then it will deserialize. It is done to save the python object on disk and converts python object into character stream.

- **seaborn**: It is python library which is used for making statistical graphics. It is build on of matplotlib library. It is used for data visualization and works easily in data frames and graphs can be customized easily.
- **WINDOWS:**
- Microsoft Windows, normally mentioned to as Windows, is a collection of numerous exclusive graphic functioning system families, entirely of which are recognized and advertised by Microsoft.

- **VISUAL STUDIO CODE:**
- "Visual studio code is a free source code editor" which supports the features like debugging, version control, task running and users can install the extensions that can add extra functionalities. It supports python fully and has useful functionalities like real-time collaboration.

5.2 MODULES DESCRIPTION:

MODULES:

Parkinsons_analysis.ipynb :

- import libraries like pandas, numpy, matplotlib.pyplot, pickle
  - Reading the dataset
  - Process the training data
  - Split into train and test data
  - Applying minmax scalar

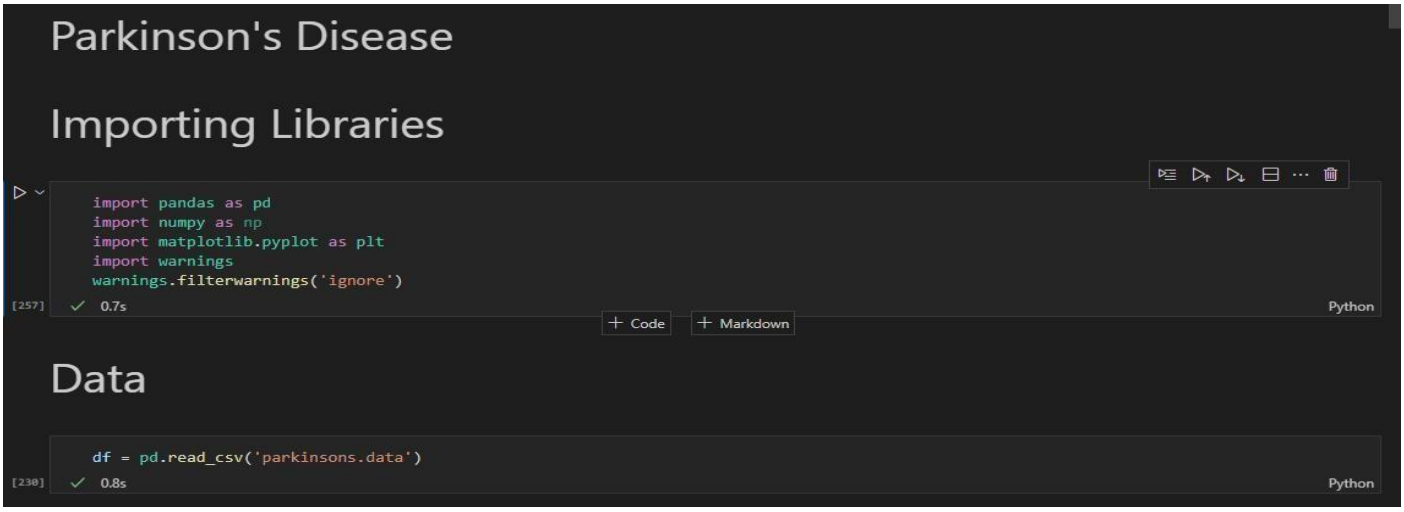


Fig 5.1 importing libraries and loading data

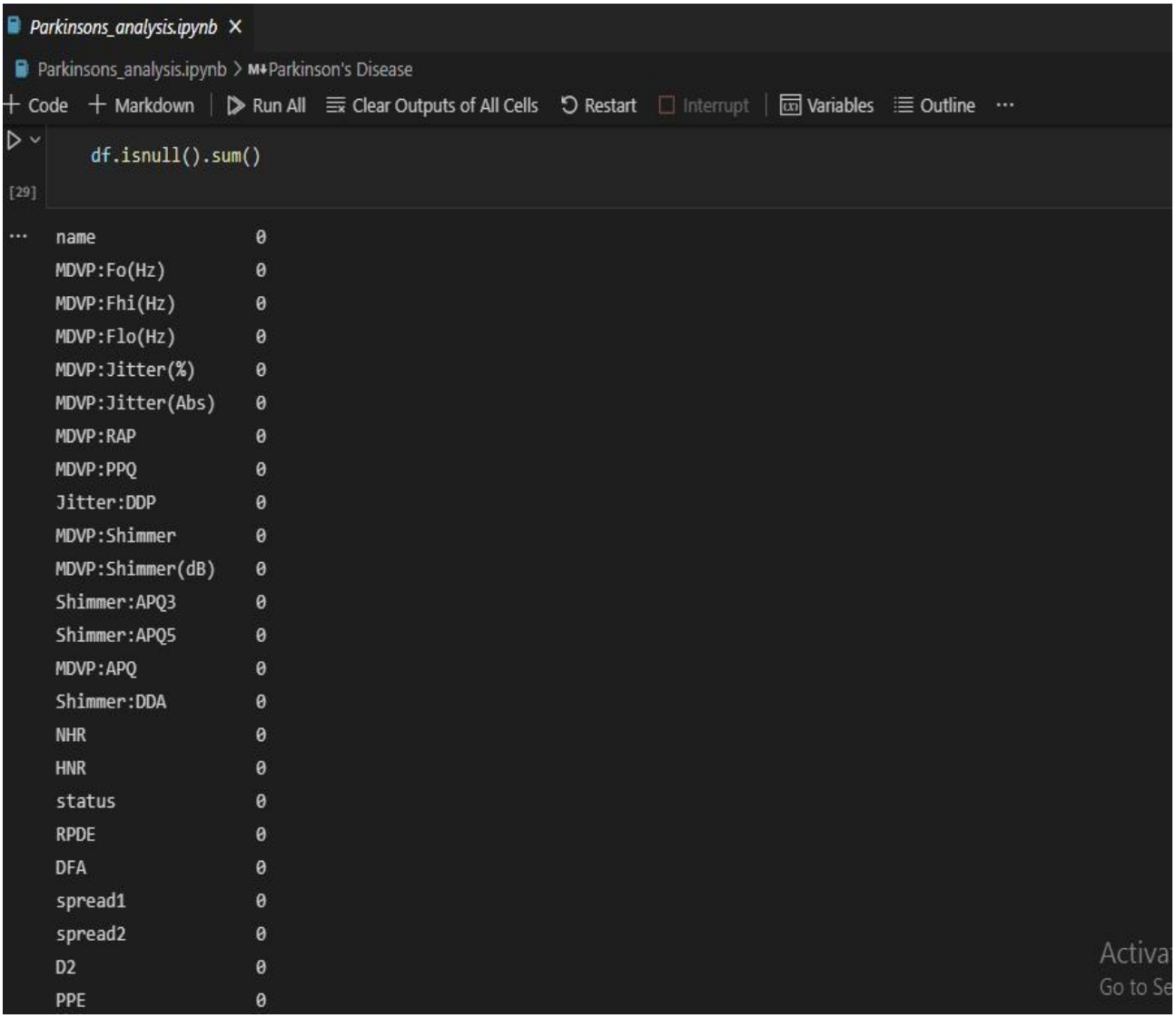


Fig 5.2 -importing libraries and loading data

```
df['status'].value_counts()
```

```
1    147
0     48
Name: status, dtype: int64
```

```
import seaborn as sns
sns.countplot(df['status'])
```

```
<AxesSubplot:xlabel='status', ylabel='count'>
```

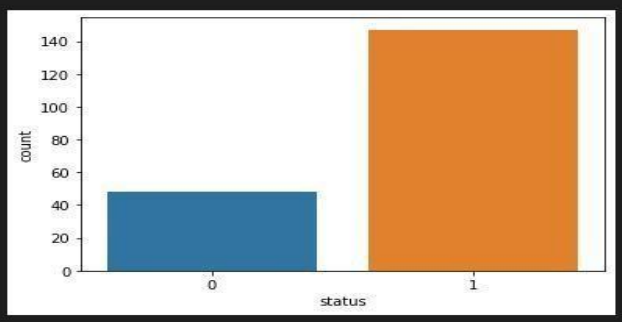


Fig 5.3-Visualization of record

Parkinsons_analysis.ipynb

Parkinsons_analysis.ipynb > Parkinson's Disease

+ Code + Markdown | Run All | Clear Outputs of All Cells | Restart | Interrupt | Variables | Outline ...

```
[34]
X = df.drop(['name'], 1)
X = X.drop(['status'], 1)
y = df['status']
```

XGB Classifier

```
[35]
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

```
[36]
from sklearn.preprocessing import MinMaxScaler
sc = MinMaxScaler(feature_range = (0,1))
```

```
[37]
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

```
from xgboost import XGBClassifier
model = XGBClassifier().fit(X_train, y_train)
```

Fig 5.4-splitting data, apply minmax scalar and xgboost classifier

```
predictions = model.predict(X_test)
```
[245] ✓ 0.7s Python

```
from sklearn.metrics import accuracy_score, f1_score, precision_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
```
[246] ✓ 0.2s Python

```
accuracy_score(y_test, predictions)
```
[247] ✓ 0.1s Python

```
0.9743589743589743
```

```
precision_score(y_test,predictions)
```
[248] ✓ 0.1s Python

```
0.967741935483871
```

```
f1_score(y_test, predictions)
```
[249] ✓ 0.2s Python

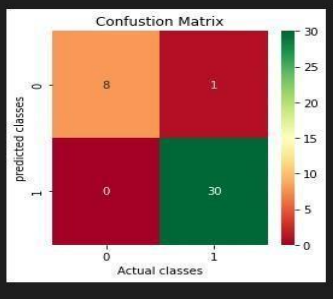
```
0.9836065573770492
```

Activate Windows
Go to Settings to activate Windows.

Fig 5.5-accuracy, precision, f1_score

```
cm=confusion_matrix(y_test,predictions)
plt.figure(figsize=(4,4))
sns.heatmap(cm,annot=True,cmap='RdYlGn',fmt='d')
plt.xlabel('Actual classes')
plt.ylabel('predicted classes')
plt.title('Confustion Matrix')
```
[256] ✓ 0.3s Python

```
Text(0.5, 1.0, 'Confustion Matrix')
```



Activate Windows

Fig 5.6 - confusion matrix

```
import pickle
# Writing different model files to file
with open( 'modelForPrediction.sav', 'wb') as f:
    pickle.dump(model,f)

with open('standardScalar.sav', 'wb') as f:
    pickle.dump(sc,f)
```
43] ✓ 0.1s Python

Activate Windows

Fig 5.7- pickle implementation

```
from sklearn import tree,metrics,ensemble,neural_network,naive_bayes,neighbors,svm
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
model= tree.DecisionTreeClassifier()
model.fit(X_train,y_train)
y_pred=model.predict(X_test)
print("decision tree Accuracy :",metrics.accuracy_score(y_test, y_pred)*100)
model= naive_bayes.GaussianNB()
model.fit(X_train,y_train)
y_pred=model.predict(X_test)
print("naive bayes Accuracy :",metrics.accuracy_score(y_test, y_pred)*100)
model= neighbors.KNeighborsClassifier(n_neighbors=5)
model.fit(X_train,y_train)
y_pred=model.predict(X_test)
print("KNN Accuracy :",metrics.accuracy_score(y_test, y_pred)*100)
model=XGBClassifier()
model.fit(X_train,y_train,eval_metric='rmse')
y_pred=model.predict(X_test)
print("XGBOOST Accuracy :",metrics.accuracy_score(y_test, y_pred)*100)
model= RandomForestClassifier(random_state=2)
model.fit(X_train,y_train)
y_pred=model.predict(X_test)
print("random forestAccuracy :",accuracy_score(y_test, y_pred)*100)
```

```
decision tree Accuracy : 87.17948717948718
naive bayes Accuracy : 74.35897435897436
KNN Accuracy : 92.3076923076923
XGBOOST Accuracy : 97.43589743589743
random forestAccuracy : 87.17948717948718
```

Activate
Go to Settir

Fig 5.8-comparision of accuracy of different algorithms

index.css and style.css

- Both “style.css” and “index.css” are “cascading style sheets”
- This two files are used for styling the web pages(index.html, results.html)

```
# index.css
static > css > # index.css > body
1   body {
2       font-family: 'lato', sans-serif;
3       font-size: 18px;
4       font-weight:bold;
5       text-align:center;
6       background-image: url('https://post.healthline.com/wp-content/uploads/2020/09/11008-Parkinsons_Disease_1200x628-faceboo
7       width: 100%;
8       height: 100%;
9       background-size: cover;
10
11  }
```

Fig 5.9-index.css

```
> css > # style.css > body
body {
    font-family: 'lato', sans-serif;
    background-color: #6C7A89;
    color: white;
    font-size: 18px;
    font-weight:bold;
    text-align:center;
}
```

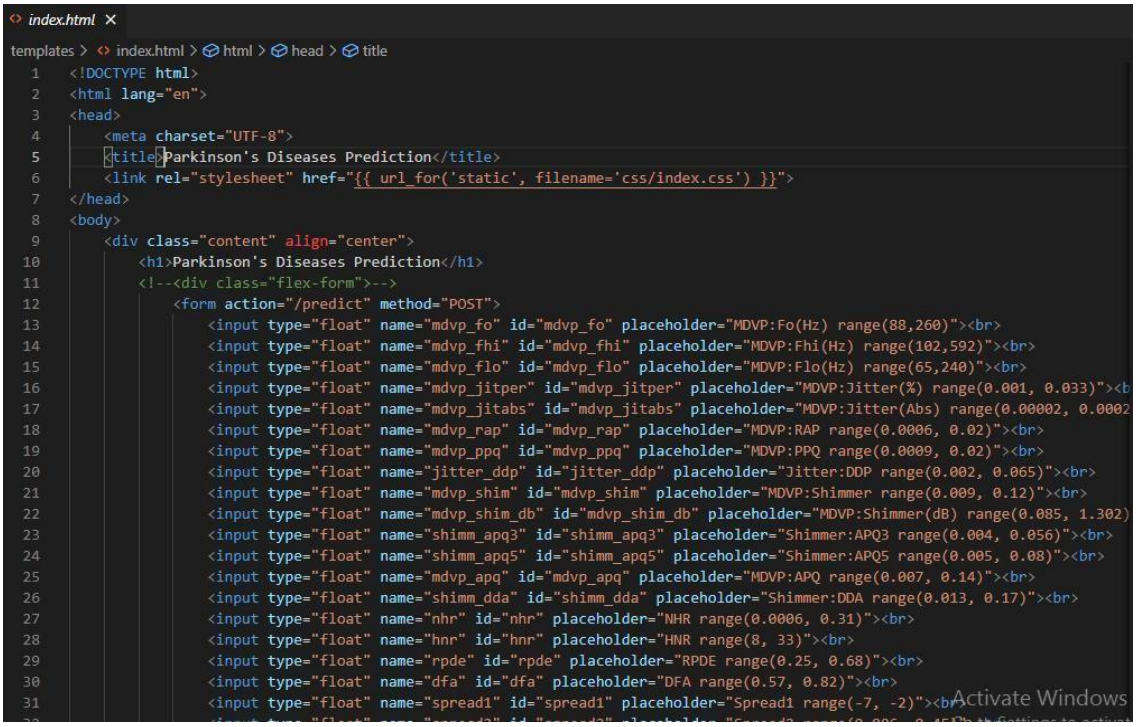
Fig 5.10-style.css

index.html and results.html

index.html

- Index.html is used to take input from user
- Both the .css files are used for styling the web pages
- The input read from this form is given to app.py for

prediction results.html

- results.html is used to display output in the webpage
- 

```
index.html
templates > index.html > html > head > title
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Parkinson's Diseases Prediction</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='css/index.css') }}">
</head>
<body>
    <div class="content" align="center">
        <h1>Parkinson's Diseases Prediction</h1>
        <!--<div class="flex-form">-->
            <form action="/predict" method="POST">
                <input type="float" name="mdvp_fo" id="mdvp_fo" placeholder="MDVP:Fo(Hz) range(88,260)"><br>
                <input type="float" name="mdvp_fhi" id="mdvp_fhi" placeholder="MDVP:Fhi(Hz) range(102,592)"><br>
                <input type="float" name="mdvp_flo" id="mdvp_flo" placeholder="MDVP:Flo(Hz) range(65,240)"><br>
                <input type="float" name="mdvp_jitper" id="mdvp_jitper" placeholder="MDVP:Jitter(%) range(0.001, 0.033)"><b
                <input type="float" name="mdvp_jitabs" id="mdvp_jitabs" placeholder="MDVP:Jitter(Abs) range(0.00002, 0.0002
                <input type="float" name="mdvp_rap" id="mdvp_rap" placeholder="MDVP:RAP range(0.0006, 0.02)"><br>
                <input type="float" name="mdvp_ppq" id="mdvp_ppq" placeholder="MDVP:PPQ range(0.0009, 0.02)"><br>
                <input type="float" name="jitter_ddp" id="jitter_ddp" placeholder="Jitter:DDP range(0.002, 0.065)"><br>
                <input type="float" name="mdvp_shim" id="mdvp_shim" placeholder="MDVP:Shimmer range(0.009, 0.12)"><br>
                <input type="float" name="mdvp_shim_db" id="mdvp_shim_db" placeholder="MDVP:Shimmer(dB) range(0.085, 1.302)
                <input type="float" name="shimm_apq3" id="shimm_apq3" placeholder="Shimmer:APQ3 range(0.004, 0.056)"><br>
                <input type="float" name="shimm_apq5" id="shimm_apq5" placeholder="Shimmer:APQ5 range(0.005, 0.08)"><br>
                <input type="float" name="mdvp_apq" id="mdvp_apq" placeholder="MDVP:APQ range(0.007, 0.14)"><br>
                <input type="float" name="shimm_dda" id="shimm_dda" placeholder="Shimmer:DDA range(0.013, 0.17)"><br>
                <input type="float" name="nhr" id="nhr" placeholder="NHR range(0.0006, 0.31)"><br>
                <input type="float" name="hnr" id="hnr" placeholder="HNR range(8, 33)"><br>
                <input type="float" name="rpde" id="rpde" placeholder="RPDE range(0.25, 0.68)"><br>
                <input type="float" name="dfa" id="dfa" placeholder="DFA range(0.57, 0.82)"><br>
                <input type="float" name="spread1" id="spread1" placeholder="Spread1 range(-7, -2)"><br>
```

Activate Windows

Fig 5.11-index.html

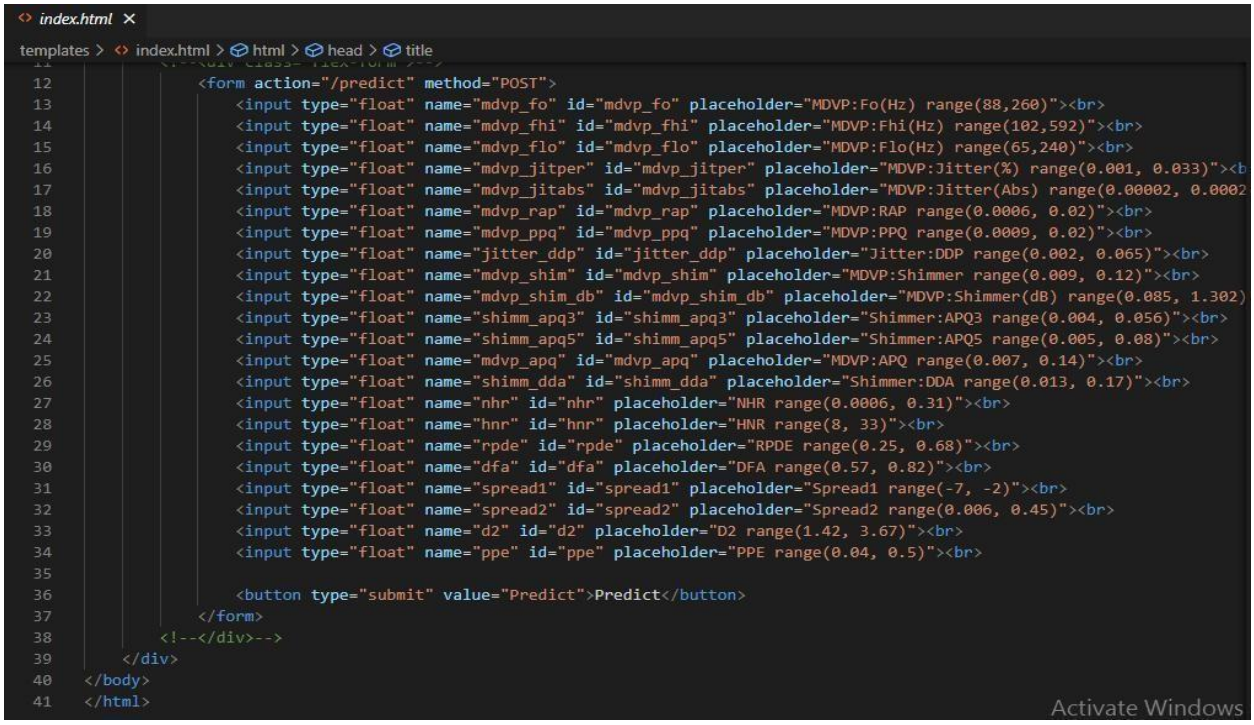


Fig 5.12-index.html

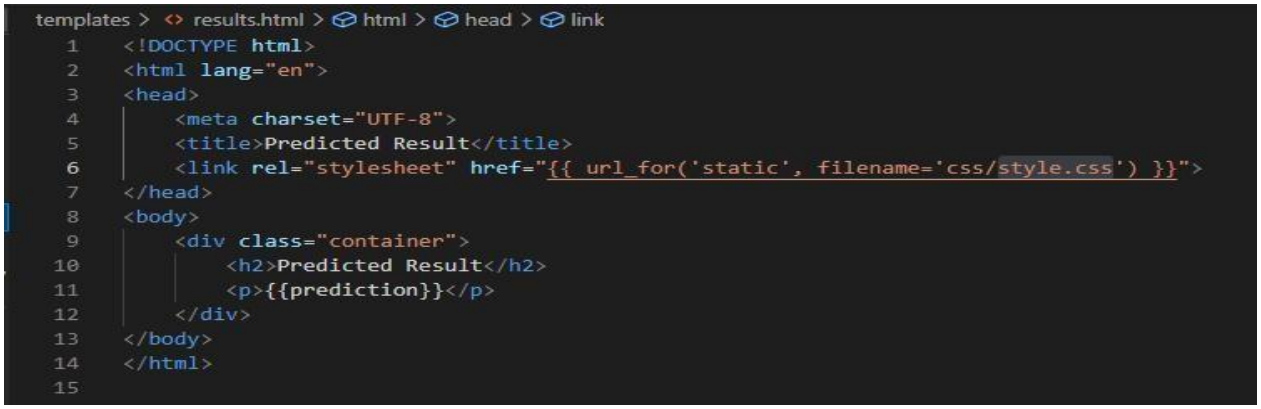


Fig 5.13-results.html

app.py

app file is used to predict whether the person is healthy or diseased using xgboost algorithm

the inputs are taken from index.html

app.py

app.py > index

```python
# importing the necessary dependencies
from flask import Flask, render_template, request
from flask_cors import CORS,cross_origin
import pickle

app = Flask(__name__) # initializing a flask app

@app.route('/',methods=['GET'])  # route to display the home page
@cross_origin()
def homePage():
    return render_template("index.html")

@app.route('/predict',methods=['POST','GET']) # route to show the predictions in a web UI
@cross_origin()
def index():
    if request.method == 'POST':
        try:
            #  reading the inputs given by the user
            mdvp_fo=float(request.form['mdvp_fo'])
            mdvp_fhi=float(request.form['mdvp_fhi'])
            mdvp_flo=float(request.form['mdvp_flo'])
            mdvp_jitper=float(request.form['mdvp_jitper'])
            mdvp_jitabs=float(request.form['mdvp_jitabs'])
            mdvp_rap=float(request.form['mdvp_rap'])
            mdvp_ppq=float(request.form['mdvp_ppq'])
            jitter_ddp=float(request.form['jitter_ddp'])
            mdvp_shim=float(request.form['mdvp_shim'])
            mdvp_shim_db=float(request.form['mdvp_shim_db'])
            shimm_apq3=float(request.form['shimm_apq3'])
            shimm_apq5=float(request.form['shimm_apq5'])
            mdvp_apq=float(request.form['mdvp_apq'])
```

Fig 5.14-app.py

app.py > ...

```python
            nhr=float(request.form['nhr'])
            hnr=float(request.form['hnr'])
            rpde=float(request.form['rpde'])
            dfa=float(request.form['dfa'])
            spread1=float(request.form['spread1'])
            spread2=float(request.form['spread2'])
            d2=float(request.form['d2'])
            ppe=float(request.form['ppe'])


            filename = 'modelForPrediction.sav'
            loaded_model = pickle.load(open(filename, 'rb')) # loading the model file from the storage
            # predictions using the loaded model file
            scaler = pickle.load(open('standardScalar.sav', 'rb'))
            prediction=loaded_model.predict(scaler.transform([[mdvp_fo,mdvp_fhi,mdvp_flo,mdvp_jitper, mdvp_jitabs,
                mdvp_rap,mdvp_ppq, jitter_ddp, mdvp_shim, mdvp_shim_db,shimm_apq3,shimm_apq5,mdvp_apq,shimm_dda,nhr,hnr,
            print('prediction is', prediction)
            if prediction == 1:
                pred = "You have Parkinson's Disease. Please consult a specialist."
            else:
                pred = "You are Healthy Person."
            # showing the prediction results in a UI
            return render_template('results.html',prediction=pred)
        except Exception as e:
            print('The Exception message is: ',e)
            return 'something is wrong'
    # return render_template('results.html')
    else:
        return render_template('index.html')
if __name__ == "__main__":
    #app.run(host='127.0.0.1', port=8001, debug=True)
```

Activate Windo

Fig 5.15-app.py

5.3 ALGORITHM

XGBOOST ALGORITHM:

- Algorithm we used right here is xgboost algorithm

a) XGBoost is a Boosting tehnique in Ensemble Learning. Ensemble learning includes a set of predictors which use multiple models for providing better prediction accuracy

b) "XGBoost" is an enhancement of "Gradient Boosting algorithm"

c) Here, decision trees are created in "sequential order".

d) "Weights" plays crucial role in "XGBoost". Weights are allocated to all independent variables which are then fed into decision tree which predicts output

e) The weights of variables predicted incorrectly by tree are increased and these variables are then fed into second decision tree. These individual classifiers then group to give a strong, more precise model

f) It works on different prediction problems like CART, ranking, user-defined problems

- WHY XGBOOST?

MAIN FEAUTURES OF GRADIENT BOOSTING

Parallel tree building –the sequential tree building done in parallelized implementation by interchanging the loops which enumerates the leaf nodes of tree.
Regularization- XGBoost has the potential to correct complicated models through "L1, L2 regularization". Regularization helps by avoiding over fitting
Tree pruning using depth-first approach XGBoost uses 'max_depth' parameter and starts pruning decision tree backwards. This "depth-first" approach significantly increases the computational performance

Consideration of cache and out-of-core calculation-this optimiszes the memory of the disk while making sure there is no continuous access of hardware and better usage of hardware and memory space by storing the data in blocks
Efficient handling of missing data-actually it is dealing with sparse data while incorporate with sparse wise split algorithm which make sure to handle different sparse data and its patterns
Built-in cross-validation capability- it has this built in method which at every iteration, the need to explicitly program this search and to mention precise number of boosting iterations needed in taking single run.

➢ PSEUDO-CODE FOR XGBOOST ALGORITHM

Input: Initializes particles $x_i = (x_i^1, x_i^2, \ldots, x_i^D)$ with
position $X_i = [x_i^1, x_i^2, \ldots, x_i^D]$ and velocity $V_i = [v_i^1, v_i^2, \ldots, v_i^D]$

Output: Optimal value of hyper-parameter

for $i = 1; i \le N; i = i + 1$ do
  Compute the local density δ_i, and the distance ρ_i;
  Choose particles with high δ_i and relatively high ρ_i as centers according to $\gamma_i = \rho_i * \delta_i$;
  Assign remaining particles and get C subgroups;
  Initialize XGBoost with instance nodes set I on train data, the hyper-parameter ← current optimal value;
  for $k = 1; k \le m; k = k + 1$ do
    $gain \leftarrow 0, G = \Sigma_{i \in I} g_i \leftarrow 0, H = \Sigma_{i \in I} g_i \leftarrow 0$
    for j *in sorted* (I, *by* x_{jk}) do
      $G_L \leftarrow G_L + g_L, H_L \leftarrow H_L + h_i$;
      $G_R \leftarrow G - G_L, H_R \leftarrow H - H_L$;
      score max (score, $\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} + \frac{G^2}{H + \lambda}$
  Update particle state ($pBest_i$, $gBest$) refer to the loss function
  for $c = 1; c \le C; c = c + 1$ do
    if *particle is local optimal then*
      $v_i^d = \omega v_i^d + c_1\, rand_1^d\, (pbest_1^d - x_i^d) + c_2\, rand_2^d\, (\frac{1}{c} \sum_{c=1} gbest_i^d - x_i^d)$
      $x_i^d = x_i^d + v_i^d$
    else
      $v_i^d = \omega v_i^d + c_1\, rand_1^d\, (pbest_1^d - x_i^d) + c_2\, rand_2^d\, (cgbest_c^d - x_i^d)$
      $x_i^d = x_i^d + v_i^d$

6.IMPLEMENTATION

6.1 Sample Code

```
# importing the necessary dependencies
from flask import Flask, render_template, request,jsonify
from flask_cors import CORS,cross_origin
import pickle

app = Flask(_name_) # initializing a flask app

@app.route('/',methods=['GET'])  # route to display the home page
@cross_origin()
def homePage():
    return render_template("index.html")

@app.route('/predict',methods=['POST','GET']) # route to show the predictions in a web UI
@cross_origin()
def index():
    if request.method == 'POST':
        try:
            #  reading the inputs given by the user
            mdvp_fo=float(request.form['mdvp_fo'])
            mdvp_fhi=float(request.form['mdvp_fhi'])
            mdvp_flo=float(request.form['mdvp_flo'])
            mdvp_jitper=float(request.form['mdvp_jitper'])
            mdvp_jitabs=float(request.form['mdvp_jitabs'])
            mdvp_rap=float(request.form['mdvp_rap'])
            mdvp_ppq=float(request.form['mdvp_ppq'])
            jitter_ddp=float(request.form['jitter_ddp'])
            mdvp_shim=float(request.form['mdvp_shim'])
            mdvp_shim_db=float(request.form['mdvp_shim_db'])
            shimm_apq3=float(request.form['shimm_apq3'])
            shimm_apq5=float(request.form['shimm_apq5'])
            mdvp_apq=float(request.form['mdvp_apq'])
            shimm_dda=float(request.form['shimm_dda'])
            nhr=float(request.form['nhr']) hnr=float(request.form['hnr'])
            rpde=float(request.form['rpde'])
            dfa=float(request.form['dfa'])


            spread1=float(request.form['spread1'])
            spread2=float(request.form['spread2'])
            d2=float(request.form['d2'])
            ppe=float(request.form['ppe'])
```

```
        filename = 'modelForPrediction.sav'
        loaded_model = pickle.load(open(filename, 'rb')) # loading the model file from the
storage
        # predictions using the loaded model file
        scaler = pickle.load(open('standardScalar.sav', 'rb'))

prediction=loaded_model.predict(scaler.transform([[mdvp_fo,mdvp_fhi,mdvp_flo,mdvp_jitper,
mdvp_jitabs,
            mdvp_rap,mdvp_ppq, jitter_ddp, mdvp_shim,
mdvp_shim_db,shimm_apq3,shimm_apq5,mdvp_apq,shimm_dda,nhr,hnr,rpde,dfa,spread1,sprea
d2,d2,ppe]]))
        print('prediction is', prediction)
        if prediction == 1:
            pred = "You have Parkinson's Disease. Please consult a specialist."
        else:
            pred = "You are Healthy Person."
        # showing the prediction results in a UI
        return render_template('results.html',prediction=pred)
    except Exception as e:
        print('The Exception message is: ',e)
        return 'something is wrong'
  # return render_template('results.html')
  else:
    return render_template('index.html')



if _name_ == "_main_":
   #app.run(host='127.0.0.1', port=8001, debug=True)
  app.run(debug=True) # running the app
@import "compass/css3";

@import
url(https://fonts.googleapis.com/css?family=Dosis:300|Lato:300,400,600,700|Roboto+Condense
d:300,700|Open+Sans+Condensed:300,600|Open+Sans:400,300,600,700|Maven+Pro:400,700);

@import  url('https://netdna.bootstrapcdn.com/font-awesome/4.0.3/css/font-awesome.css');



* {
 @include box-sizing(border-box);
}
```

```
html {
  background: black;
}

body {
  font-family: "Open Sans";
  font-size: 16px;
  color: White;
  font-smoothing: antialiased;
  font-weight: 600;
}

.content:before
  { content: "";
  position: fixed;
  left: 0;
  right: 0;
  top: 0; bottom: 0;
  z-index: -1;

  display: block;
  background-color: black;
  background-image: url('https://post.healthline.com/wp-content/uploads/2020/09/11008-
Parkinsons_Disease_1200x628-facebook-1200x628.jpg');
  width: 100%;
  height: 100%;
  background-size: cover;

  -webkit-filter: blur(2px);
  -moz-filter: blur(2px);
  -o-filter: blur(2px);
  -ms-filter: blur(2px);
  filter: blur(2px);
}

.content {
  position: absolute;
  top: 0; bottom: 0; left: 0; right: 0;
  width: 540px;
  height: 540px;

  background-color: rgba(10, 10, 10, 0.5);
  margin: auto auto;
  padding: 40px;
  text-align: center;
```

```
  @include border-radius(4px);
  @include box-shadow(0 0 10px rgba(0,0,0,1));
 }

h1 {

    text-align: center;
    font-size: 2.0rem;
    font-weight:  600;
    color: White;
    padding-bottom: 30px;

}

input {
    width: 80%;
    font-size: 1.2rem;
    font-family: "Open Sans";

    margin: 10px 0px;

    border: none;
    padding: 10px;

    @include border-radius(4px);

}

button {
    width: 80%;
    font-size: 1.1rem;
    padding: 10px;
    margin: 20px 0px;

    background-color: #66A756;
    color: White;
    border: none;
    border-radius: 25px
    @include border-radius(4px);
}
body {
    font-family: 'lato', sans-serif;
    background-color: #6C7A89;
    color: white;
```

```
    font-size: 18px;
    font-weight:bold;
    text-align:center;

}
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Parkinson's Diseases Prediction</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='css/main.css') }}">
</head>
<body>
    <div class="content">
        <h1>Parkinson's Diseases Prediction</h1>
        <!--<div  class="flex-form">-->
            <form action="/predict" method="POST">
                <input type="float" name="mdvp_fo" id="mdvp_fo" placeholder="MDVP:Fo(Hz)
range(88,260)">
                <input type="float" name="mdvp_fhi" id="mdvp_fhi" placeholder="MDVP:Fhi(Hz)
range(102,592)">
                <input typerange(65,240)">
                <input type="float" name="mdvp_jitper" id="mdvp_jitper"
placeholder="MDVP:Jitter(%) range(0.001, 0.033)">
                <input type="float" name="mdvp_jitabs" id="mdvp_jitabs"
placeholder="MDVP:Jitter(Abs) range(0.00002, 0.0002)">
                <input type="float" name="mdvp_rap" id="mdvp_rap" placeholder="MDVP:RAP
range(0.0006, 0.02)">
                <input type="float" name=″float" name="mdvp_flo" id="mdvp_flo"
placeholder="MDVP:Flo(Hz)
="mdvp_ppq" id="mdvp_ppq" placeholder="MDVP:PPQ range(0.0009, 0.02)">
                <input type="float" name="jitter_ddp" id="jitter_ddp" placeholder="Jitter:DDP
range(0.002, 0.065)">
                <input type="float" name="mdvp_shim" id="mdvp_shim"
placeholder="MDVP:Shimmer range(0.009, 0.12)">
                <input type="float" name="mdvp_shim_db" id="mdvp_shim_db"
placeholder="MDVP:Shimmer(dB) range(0.085, 1.302)">
                <input type="float" name="shimm_apq3" id="shimm_apq3"
placeholder="Shimmer:APQ3 range(0.004, 0.056)">
                <input type="float" name="shimm_apq5" id="shimm_apq5"
placeholder="Shimmer:APQ5 range(0.005, 0.08)">
                <input type="float" name="mdvp_apq" id="mdvp_apq" placeholder="MDVP:APQ
range(0.007, 0.14)"> range(0.013, 0.17)">
                <input type="float" name="nhr" id="nhr" placeholder="NHR range(0.0006, 0.31)">
                <input type="float" name="hnr" id="hnr" placeholder="HNR range(8, 33)">
                <input type="float" name="rpde" id="rpde" placeholder="RPDE range(0.25, 0.68)">
```

```
            <input type="float" name="dfa" id="dfa" placeholder="DFA range(0.57, 0.82)">
            <input type="float" name="spread1" id="spread1" placeholder="Spread1 range(-7, -
2)">
            <input type="float" name="spread2" id="spread2" placeholder="Spread2 range(0.006,
0.45)">
            <input type="float" name="d2" id="d2" placeholder="D2 range(1.42, 3.67)">
            <input type="float" name="ppe" id="ppe" placeholder="PPE range(0.04, 0.5)">

            <input type="float" name="shimm_dda" id="shimm_dda"
placeholder="Shimmer:DDA

             <button type="submit" value="Predict">Predict</button>
          </form>
       <!--</div>-->
   </div>
</body>
</html>
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <title>Predicted Result</title>
   <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
</head>
<body>
   <div class="container">
      <h2>Predicted Result</h2>
      <p>{{prediction}}</p>
   </div>
</body>
</html>
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <title>Predicted Result</title>
   <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
</head>
<body>
   <div class="container">
      <h2>Predicted Result</h2>
      <p>{{prediction}}</p>
   </div>
   </body>
   </html>
```

6.2 UDERS SCREENS

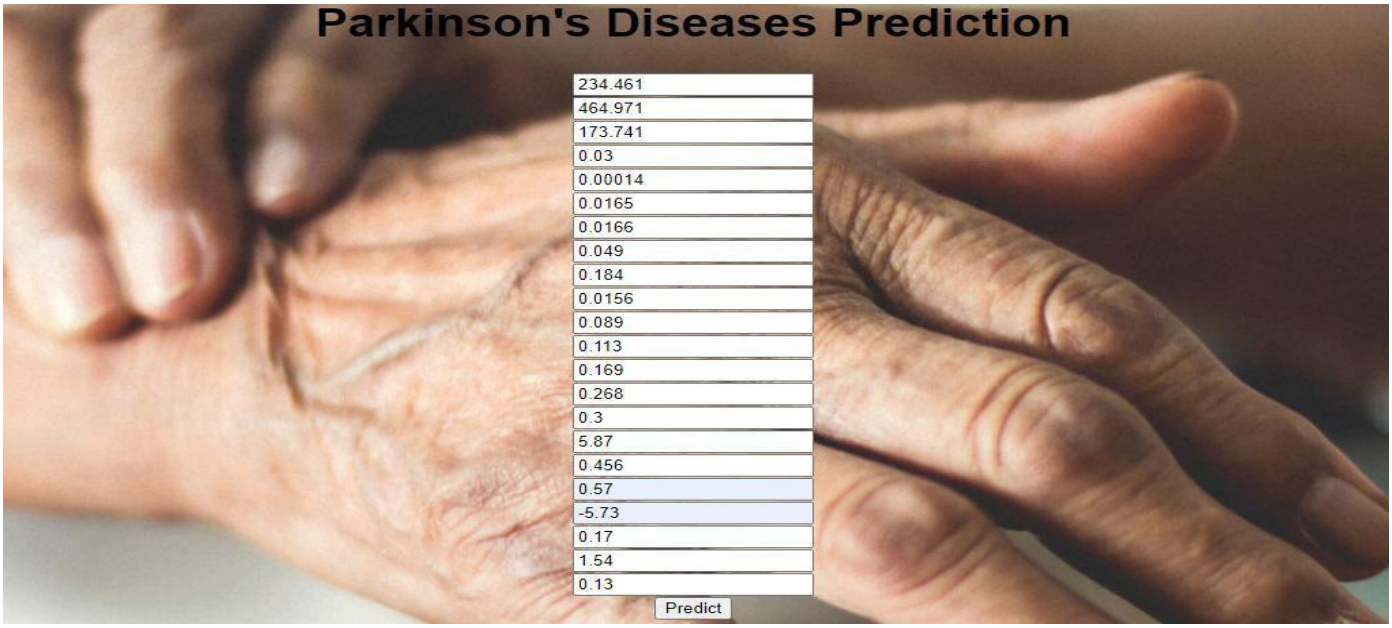


Fig 6.1-user input values



Fig 6.2 predicted output for a healthy person



Fig 6.3-user input values



Fig 6.4 Predicted output for a Parkinson's person

7.TESTING METHODLOGY

The testing methodology outlines a structured approach used to test the Flask-based plant detection system to ensure functionality, accuracy, performance, and user experience.
to identify defects, validate the deep learning model, and ensure a seamless user experience.

7.1 TYPES OF TESTING

1. MANUAL TESTING:

Definition: Testing the system manually by executing test cases without automation tools.

Purpose: To validate the web interface, usability, and core functionalities .

Focus Areas:

o Ensuring all UI elements (buttons, file upload, result display) function correctly.

o Verifying the flow of image data between the frontend and backend.

o Manually testing correct and incorrect inputsto check error handling.

1. AUTOMATED TESTING:

Definition: Using automated scripts and tools to perform repetitive and regression testing.

Purpose: To ensure system stability and identify performance bottlenecks.

Focus Areas:

API Testing: Ensuring the Flask API correctly processes image uploads model.

Regression Testing: Rerunning tests after changes to verify that new updates

2 .FUNCTIONAL TESTING

Definition: Validating that the system performs disease detection correctly according .

Purpose: To ensure the system meets functional requirements.

Approach:

Test valid and invalid image uploads.Ensure disease classification is accurate

(healthy leaf should not be misclassified as diseased).

o Verify that the Flask application correctly retrieves and displays disease details.

o Ensure real-time weather data is fetched properly when a user enters a city name.39

3.NON-FUNCTIONAL TESTING

Definition: Testing system attributes not related to functionality, such as performance.

Purpose: To validate speed, reliability, and user experience.

Key Types:

Performance Testing: Measure how long it takes for an image to be processed and classified.

o **Load Testing**: Simulate multiple users uploading images to check for slowdowns.

o **Usability Testing**: Ensure the web interface is user-friendly and easy to navigate.

Security Testing: Validate API security, protect against malicious file uploads, and secure weather API requests.

4.BLACK BOX TESTING

Definition: Testing the external system behavior without knowledge of internal workings.

Purpose: To focus on inputs and expected outputs.

Focus Areas:

File Handling: Ensuring users can upload valid image files, and the system.

Disease Detection: Checking if the CNN model correctly predicts plant diseases.

Output Handling: Verifying that disease classification and weather-based recommendations are displayed correctly.

5.WHITE BOX TESTING

Definition: Testing the internal structure, logic, and code flow.

Purpose: To verify how the CNN model, Flask API, and database interact.

Focus Areas:

Model Predictions: Ensuring the CNN model loads correctly .

API Requests: Testing how the Flask API handles requests for image uploads and weather data.

Error Handling: Checking how the system handles unexpected inputs or failures (e.g., corrupted images, missing API responses).40

7.2TESTING PHASES

1.UNIT TESTING

Definition: Testing individual components in isolation.

Goal: Ensure that each function (image preprocessing, model inference, API calls) works correctly.

Tools: Python's pytest or unittest for backend testing.

2.INTEGRATION TESTING

Definition: Testing how different system components interact.

Goal: Validate the connection between:

The Flask web application and the CNN model.

The Flask API and the OpenWeather API.

1.SYSTEM TESTING

Definition: Testing the entire Flask-based application as a whole.

Goal: Validate that all functionalities, from image upload to disease prediction and plant .

1.ACCEPTANCE TESTING

Definition: Testing the system from an end-user perspective.

Goal: Ensure the application meets user needs by handling real-world use ,

Test Case ID	Test Scenario	Test Steps	Expected Result
TC_01	Verify the application launch	Open the application in a browser	Application should load successfully
TC_02	Verify user input form	Enter patient details (voice attributes) in the form	Data should be accepted without error
TC_03	Validate missing input	Leave one or more fields blank and submit	Application should show validation error
TC_04	Verify incorrect input format	Enter text instead of numerical values	Application should show a format error
TC_05	Check prediction result	Submit valid input	System should classify as "Healthy" or "Parkinson's Detected"
TC_06	Check prediction accuracy	Compare predicted result with actual diagnosis	Model should be accurate within expected limits
TC_07	Validate UI responsiveness	Open the application on different screen sizes	Application should adjust correctly
TC_08	Verify error handling for invalid data	Enter negative values in input fields	Application should reject invalid values
TC_09	Check database integration	Submit input and check if it's saved in the database	Data should be stored successfully
TC_10	Verify performance	Submit multiple requests simultaneously	Application should respond within an acceptable time
TC_11	Validate security	Check if sensitive data (patient details) is encrypted	Data should be securely stored and transmitted
TC_12	Verify model accuracy comparison	Compare XGBoost model with other algorithms	XGBoost should provide the highest accuracy
TC_13	Check browser compatibility	Run application in Chrome, Firefox, Edge, and Safari	Application should work correctly on all
TC_14	Test model training process	Train the model with new data	Model should update and improve accuracy
TC_15	Validate deployment	Deploy model on a server and access remotely	Model should work without issues

fig 7 Test cases

2.CONCLUSION&FUTURE ENCHANCEMENT

XGBOOST technique achieves maximum accuracy and low error rate when compared to other classifiers in detection of Parkinson's disease. Someone showing degree one among parkinson's might additionally experience moderate symptoms that usually do now not intrude with daily activities , tremors and modifications in facial and strolling expressions , regardless of how lengthy it takes to attain stage four of five someone with Parkinson's the man or women will get signs and symptoms that grow to be weakening if any early detection approach of parkinson's disease can be made , it could grow to be an empowering issue for patients because the patients can begin with the modern-day remedies earlier than the damaging progression of the disorder. Parkinson's sickness cannot be cured however medicinal drug and right physiological remedies may assed control the signs dramatically . With or machine learning system, the detection of PD may be made before it progresses to immoderate levels . Contributing to the developing of medical technologies , this venture might also assist the patient as well as the scientific assistance to come upon Parkinson's in advance and with early treatment , they may also get additional time to make enhance directives and setup long lasting attorney to ensure their health care needs to be observed .rough the use of Flask for backend development, PyTorch for deep learning, and Pandas for structured data handling, the system provides a seamless and interactive experience. The web-based interface ensures that users can easily upload plant images, receive disease predictions, and access preventive measures and treatment suggestions. Overall, this project highlights the potential of artificial intelligence in agriculture, offering a practical and scalable approach to early plant disease detection and prevention.

9.REFERENCES

1. Early Detection of Parkinson's Disease Using Machine Learning: A Review

Reference: Umer, M., & Alam, M. S. (2020). Early Detection of Parkinson's Disease Using Machine Learning: A Review.

Link: ResearchGate

Summary: This paper provides a comprehensive review of various machine learning approaches used for early detection and diagnosis of Parkinson's disease. It discusses several techniques, including support vector machines (SVM), decision trees, and neural networks.

2. Parkinson's Disease Diagnosis Using Machine Learning: A Survey

Reference: Elakkiya, R., & Manikandan, S. (2021). Parkinson's Disease Diagnosis Using Machine Learning: A Survey.

Link: Springer Link

Summary: This survey discusses various machine learning algorithms used in Parkinson's disease diagnosis, including classification models like Random Forests, Naive Bayes, and K-Nearest Neighbors (KNN).

3. Machine Learning for Parkinson's Disease Diagnosis and Progression Prediction Reference: Zhang, J., & Liu, X. (2021). Machine Learning for Parkinson's Disease Diagnosis and Progression Prediction.

Link: Elsevier ScienceDirect

Summary: This paper focuses on both diagnosis and the prediction of disease progression in Parkinson's disease using machine learning models, particularly regression models and deep learning techniques.

4. Classification of Parkinson's Disease Using Random Forest

Reference: Kira, Z., & Pathan, S. M. (2020). Classification of Parkinson's Disease Using Random Forest.

Link: IEEE Xplore

Summary: This work discusses the use of the Random Forest classification algorithm for the prediction of Parkinson's disease using clinical data. It highlights the efficiency of Random

Forests in handling complex and noisy data.5. Parkinson's Disease Detection Using Speech and Motor Control Features

Reference: Tripathi, S., & Sharma, S. (2019). Parkinson's Disease Detection Using Speech and Motor Control Features

Link: SpringerLink

Summary: This paper focuses on combining speech and motor control data for Parkinson's detection. It uses various machine learning techniques for feature extraction and classification.